Latent Variable Models

Evaluation & Modification

Agenda

- Estimating CFAs
- Evaluating a single model
  - Chi-square
  - Fit indices
- Comparing multiple models
  - Nested Models
  - Chi-square test of model difference
  - AIC and BIC

Adolescent Depression during COVID-19

Schleider, J.L., Mullarkey, M.C., Fox, K.R. *et al.* A randomized trial of online single-session interventions for adolescent depression during COVID-19. *Nat Hum Behav* **6**, 258–268 (2022). https://doi.org/10.1038/s41562-021-01235-0

Participants (N = 2,452), age 13-16, with elevated depression are randomly assigned to one of three conditions (all single session online interventions):

1. Behavioral Activation
2. Growth Mindset
3. Control

At baseline and follow-up (3 mo.) participants are measured on:

- Hopelessness
- Agency
- Depression
- Generalized Anxiety
- COVID-19 related trauma
- Restrictive Eating

AI Image generated with paper title from Hotpot.ai

Depression Measure

Depression is measured using the Childhood Depression Inventory – Short Form (CDI-SF)

People sometimes have different feelings and ideas.

This section lists the feelings and ideas in groups. From each group of three sentences, pick one sentence that describes you best for the **past two weeks**. After you pick a sentence from the first group, go on to the next group.

There is no right or wrong answer. Just pick the sentence that best describes the way you have been feeling recently.

Some example items:

Pick the sentence that best describes the way you have been feeling for the **past two weeks**.

○ I am sad once in a while.	○ Nothing will ever work out for me.	○ I do most things okay.	○ I have fun in many things.
○ I am sad many times.	○ I am not sure if things will work out for me.	○ I do many things wrong.	○ I have fun in some things.
○ I am sad all the time.	○ Things will work out for me okay.	○ I do everything wrong.	○ Nothing is fun at all.

Try loading in the data

I've provided the dataset and an R script:

- schleider2022data.rds
- CFAModelsSyntax.R

You can save these in the same folder and start and R project in that space (recommended)

Alternatively set your working directory to wherever the data is prior to running the script

```
setwd(dir)
```

Estimation

Maximum Likelihood Estimation

- Just as in the observed variable models we are going to use maximum likelihood estimation.
- Based on the assumption that the observed variables (and the latent variables) are multivariate normally distributed…

$$\boldsymbol{Z} \sim MVN(\mathbf{0}, \boldsymbol{\Sigma}(\boldsymbol{\theta}))$$

We can define a likelihood for the $\boldsymbol{Z}$s and define a log-likelihood for the $\boldsymbol{Z}$s

$$f(\boldsymbol{Z}) \qquad LL(\boldsymbol{Z}) = \log(f(\boldsymbol{Z}))$$

The fitting function is the same as with the observed variable data:

$$F_{ML} = \log|\boldsymbol{\Sigma}(\boldsymbol{\theta})| + tr(\boldsymbol{S}\boldsymbol{\Sigma}(\boldsymbol{\theta})^{-1}) - \log|\boldsymbol{S}| - (N_Z)$$

We want to find $\boldsymbol{\theta}$ which **minimizes** the fit function.

Properties of Maximum Likelihood

- Estimates are asymptotically unbiased
- Estimates are consistent
- Estimates are asymptotically efficient
  - Among all consistent estimators none has a smaller asymptotic variance
  - Gives a standard error estimate
- Estimators are asymptotically normal
  - Allows us to use estimate, SE, to get Z-statistic, and *p*-value

Other useful properties:

- Scale invariant (rescaling variables doesn't change fit)
- Scale free (rescaling variables changes coefficients in a predictable way)

Under the Hood of ML

Last time we talked about taking derivatives and second derivatives setting those to certain values and solving for parameters to get estimates.

This is not actually what the computer does.

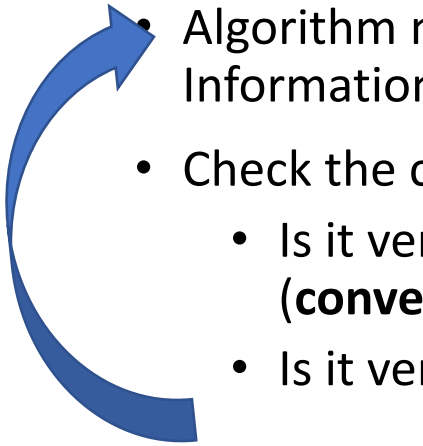
It uses **iterative optimization algorithms.**

Why do you need to know about this?

- Many error messages refer to issues with this process, so you need to know how to diagnose.
- Many settings in lavaan refer to parts of this process, so you need to know what those are.
- Extensions of SEM (missing data, categorical data, robust standard errors) involve adjustments of different parts of this process

Estimation Steps

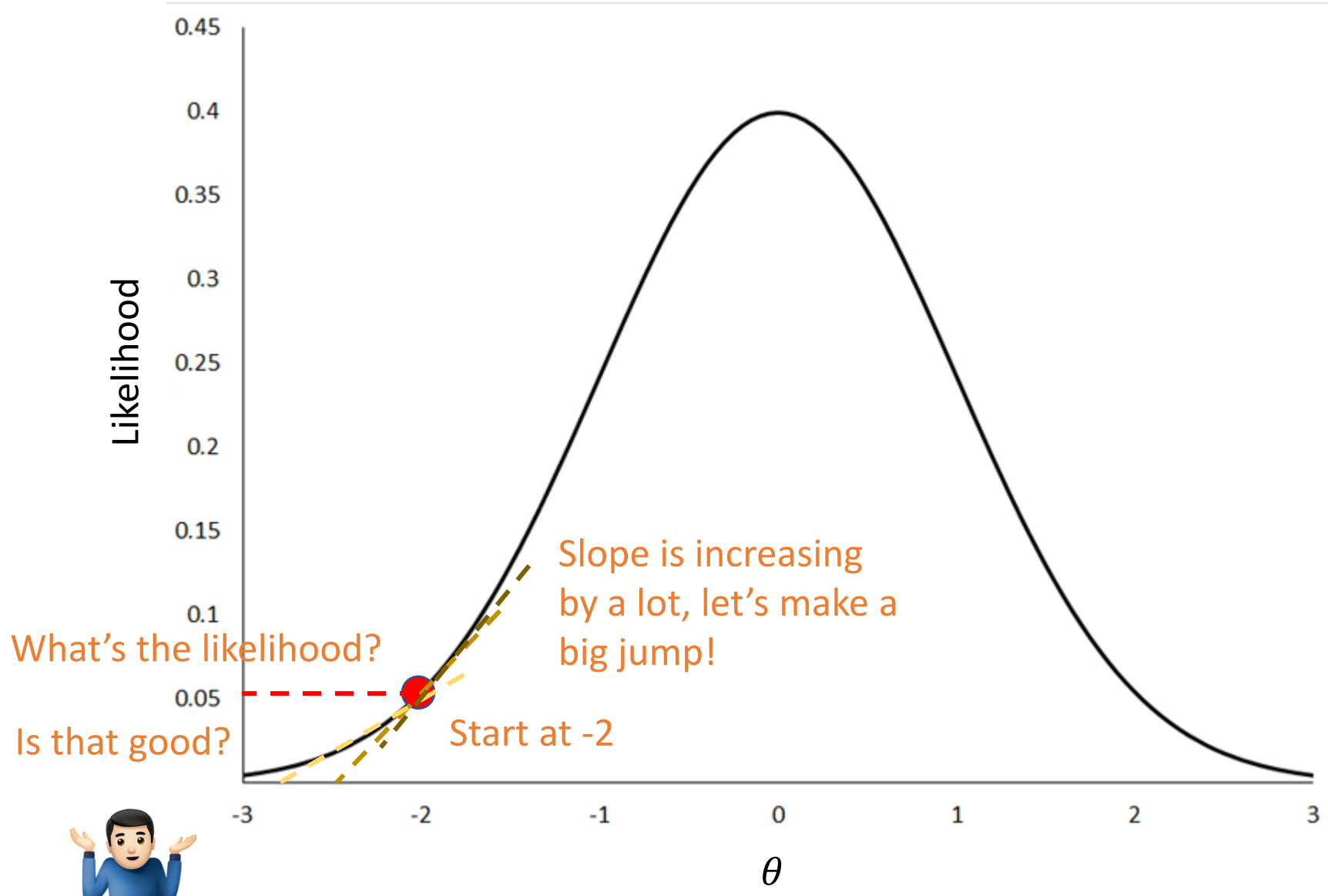
- Program begins with an initial guess about the parameters (**start values**)
- Fit function and Information Matrix estimated at those values
  - Fit function tells us how good we're doing, Information Matrix tells us what direction to go
- Algorithm moves estimates in the direction predicted (based on Information Matrix) to improve the fit function (step = step+1)
- Check the change in the likelihood
  - Is it very small? (within our **tolerance**)? Then we're done (**convergence**)!
  - Is it very large? (more than our **tolerance**)? Step again!



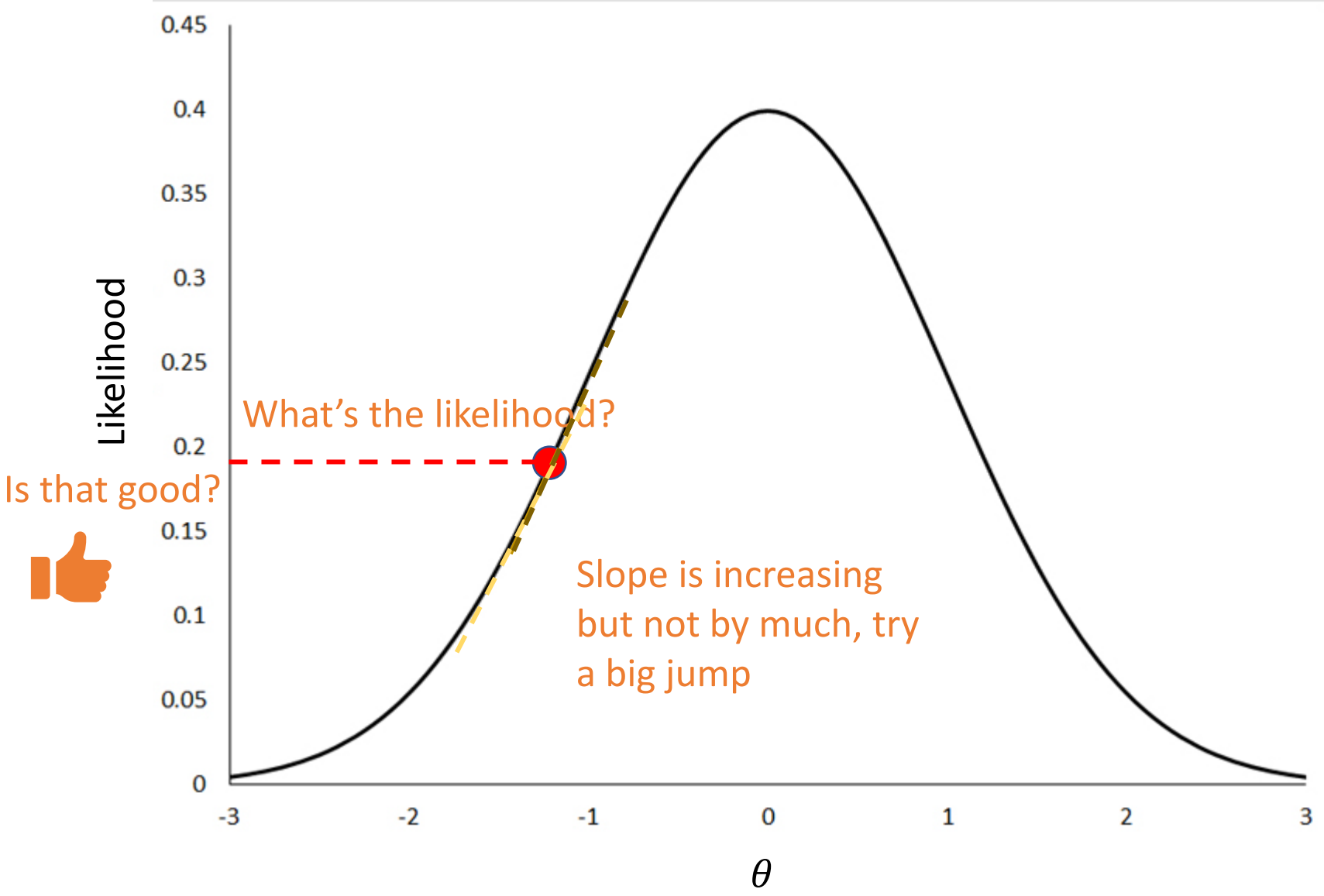
Repeat until convergence or we hit our maximum number of **iterations**

If we hit maximum number of **iterations** without convergence, we call this **nonconvergence**

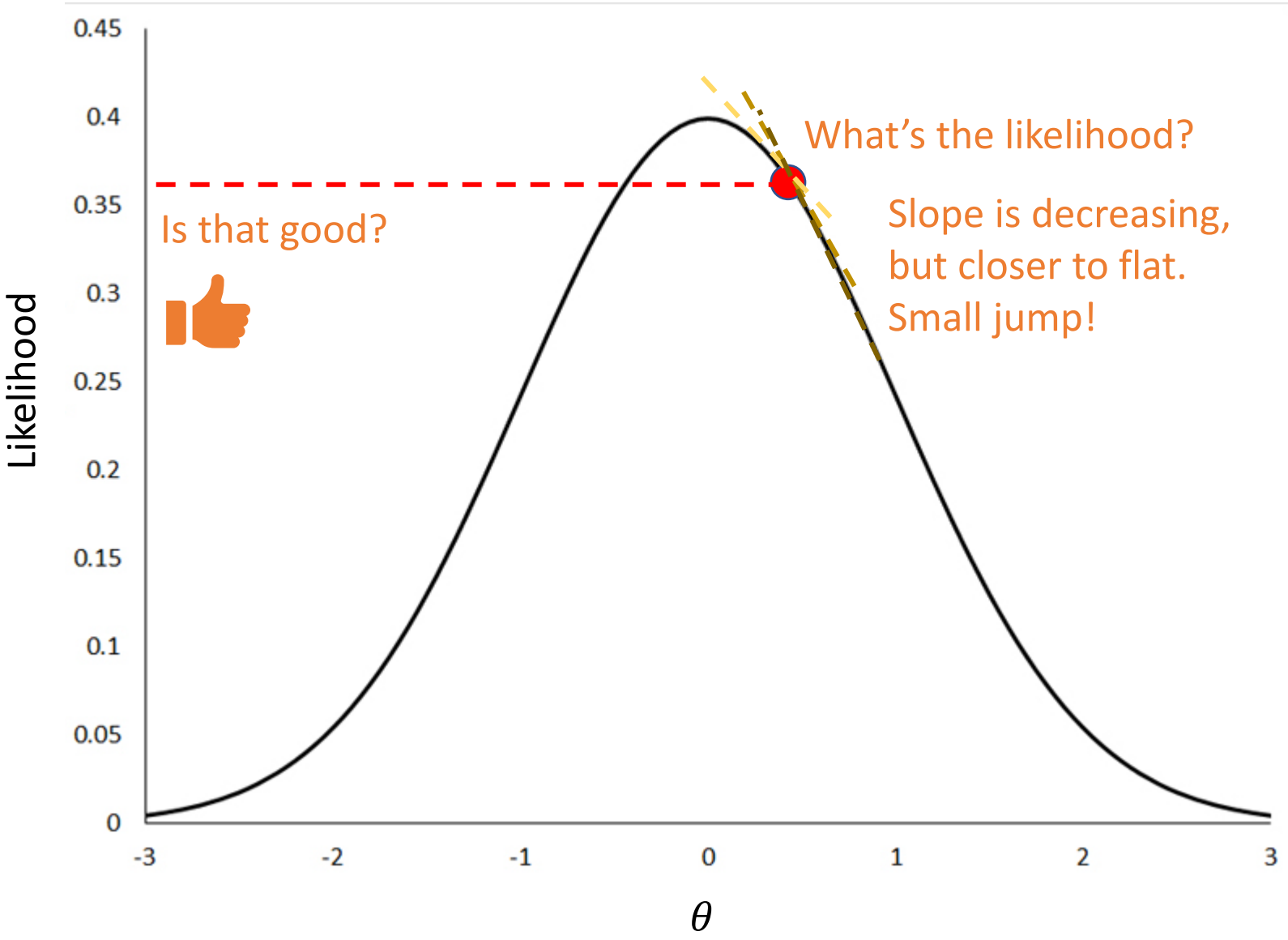
A visualization



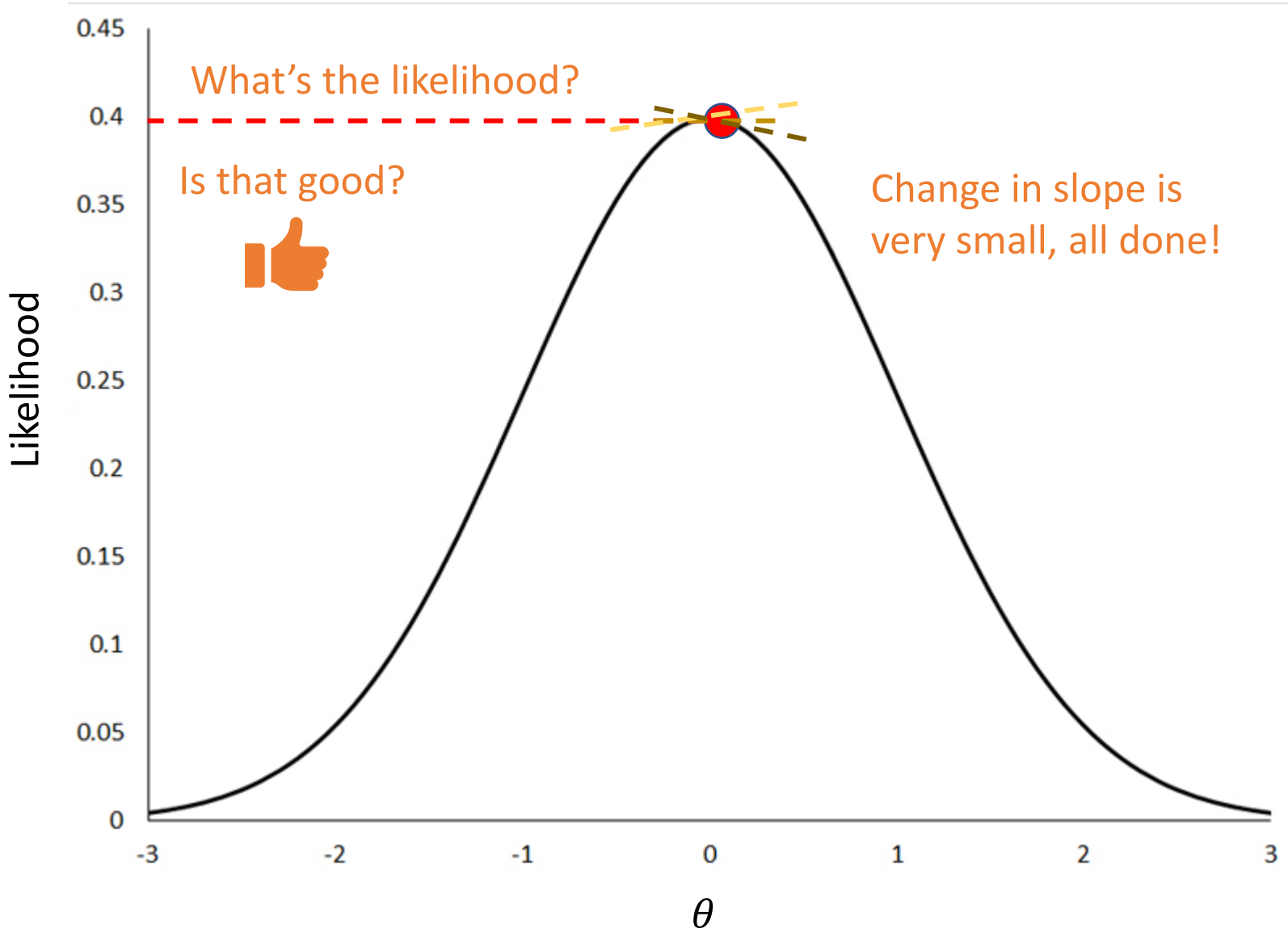
A visualization



A visualization



A visualization



How big do we jump?

Big Jump

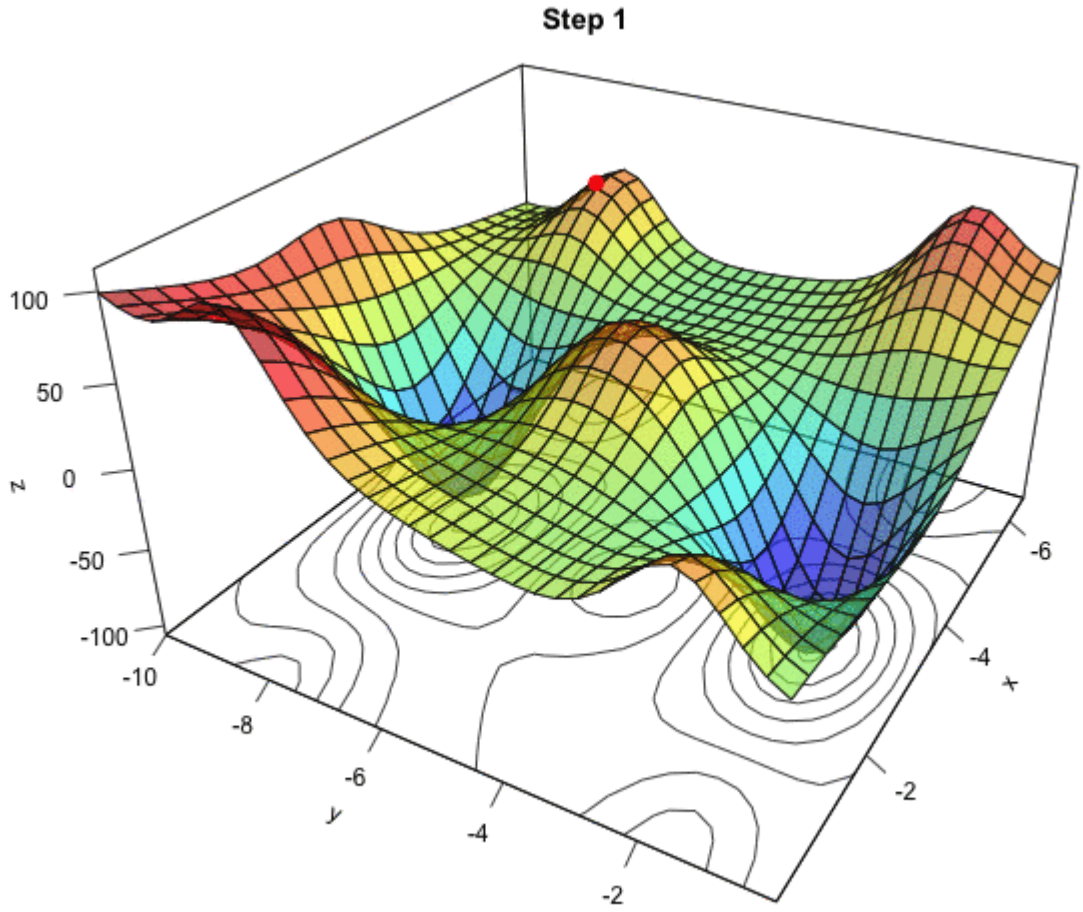
- When slope is changing rapidly
- When slope is high

Small Jump

- When slope is changing slowly
- When slope is small

Some programs add randomness to their jump size to improve convergence rates.

Closer to reality



Likelihood surfaces can be very complex for structural equation models

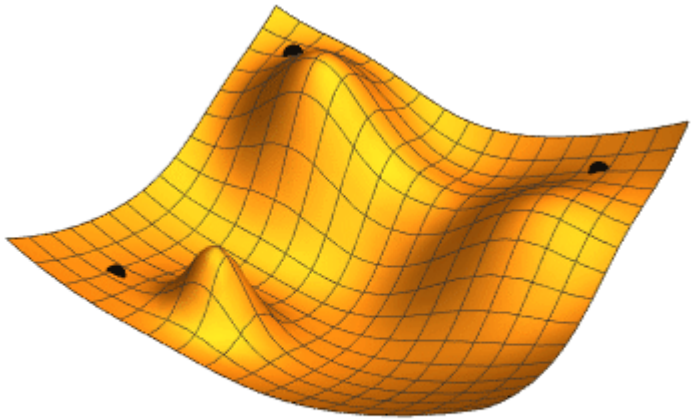
Multiple Parameters

- When we have multiple parameters (so pretty much all the time) the algorithm is going to hold the other parameters constant and calculate the **gradient** for each parameter
  - **Gradient:** The estimated derivative of the likelihood function for a given parameter

- Gradients for each parameter are calculated, then a step is made in multidimensional space based on these

Start Values

- Start values can help with convergence
  - If you start in a low likelihood flat area, you may never escape
- Trying multiple start values can help ensure you did not find a local minimum instead of a global minimum
- Inputting start values can help with convergence
  - Start on a higher point of the hill
- **Note:** This tends to only matter for somewhat complex models (e.g., multiple group models, models with categorical predictors)



Start Values

Most SEM programs start with somewhat "silly" start values

`sem(...,start = "simple")`

- All parameters set to 0, except…
- Factor loadings = 1
- Latent variable variances = .05
- Residual variances = observed variable variances

`sem(...,start = "Mplus")`

- Mplus has some "fancy" built in settings to help select good start values
- Factor loadings estimated with two-stage least squares

`sem(...,start = fit)`

- Where fit is a lavaan object, it will use the solutions from that object as starting values for the new model

Let's Try Some Estimation!

Open the R script. Load in the data and run first model (through Line 12)

```
measurementModelcfa <- `
      CDI_fu =~f1_cdi_1+f1_cdi_2+f1_cdi_3+f1_cdi_4
             +f1_cdi_5+f1_cdi_6+f1_cdi_7+f1_cdi_8
             +f1_cdi_9+f1_cdi_10+f1_cdi_11
             +f1_cdi_12
`

measureFitcfa <- cfa(measurementModelcfa, data =
cdidat)

summary(measureFitcfa, fit.measures=TRUE)
```

Try Different Starting Values

Try running the cfa function with “simple” (default) and “Mplus” do any of the estimate differ?

Simple: `lavaan 0.6-12 ended normally after 32 iterations`

```
Latent Variables:
                   Estimate  Std.Err  z-value  P(>|z|)
  CDI_fu =~
    f1_cdi_1          1.000
    f1_cdi_2          0.823    0.037   21.990    0.000
    f1_cdi_3          0.976    0.042   23.193    0.000
    f1_cdi_4          0.636    0.033   19.046    0.000
    f1_cdi_5          0.879    0.046   19.233    0.000
    f1_cdi_6          1.106    0.045   24.427    0.000
    f1_cdi_7          0.638    0.037   17.159    0.000
    f1_cdi_8          0.687    0.038   17.903    0.000
    f1_cdi_9          0.625    0.042   14.902    0.000
    f1_cdi_10         0.820    0.043   19.099    0.000
    f1_cdi_11         0.789    0.047   16.725    0.000
    f1_cdi_12         0.931    0.042   21.987    0.000

Variances:
                   Estimate  Std.Err  z-value  P(>|z|)
   .f1_cdi_1          0.235    0.010   23.358    0.000
   .f1_cdi_2          0.222    0.009   24.490    0.000
   .f1_cdi_3          0.256    0.011   23.860    0.000
   .f1_cdi_4          0.212    0.008   25.543    0.000
   .f1_cdi_5          0.394    0.015   25.491    0.000
   .f1_cdi_6          0.264    0.011   23.011    0.000
   .f1_cdi_7          0.288    0.011   25.995    0.000
   .f1_cdi_8          0.297    0.011   25.832    0.000
   .f1_cdi_9          0.399    0.015   26.396    0.000
   .f1_cdi_10         0.350    0.014   25.529    0.000
   .f1_cdi_11         0.472    0.018   26.083    0.000
   .f1_cdi_12         0.285    0.012   24.491    0.000
    CDI_fu            0.220    0.015   14.479    0.000
```

Mplus: `lavaan 0.6-12 ended normally after 32 iterations`

```
Latent Variables:
                   Estimate  Std.Err  z-value  P(>|z|)
  CDI_fu =~
    f1_cdi_1          1.000
    f1_cdi_2          0.823    0.037   21.990    0.000
    f1_cdi_3          0.976    0.042   23.193    0.000
    f1_cdi_4          0.636    0.033   19.046    0.000
    f1_cdi_5          0.879    0.046   19.233    0.000
    f1_cdi_6          1.106    0.045   24.427    0.000
    f1_cdi_7          0.638    0.037   17.159    0.000
    f1_cdi_8          0.687    0.038   17.903    0.000
    f1_cdi_9          0.625    0.042   14.902    0.000
    f1_cdi_10         0.820    0.043   19.099    0.000
    f1_cdi_11         0.789    0.047   16.725    0.000
    f1_cdi_12         0.931    0.042   21.987    0.000

Variances:
                   Estimate  Std.Err  z-value  P(>|z|)
   .f1_cdi_1          0.235    0.010   23.358    0.000
   .f1_cdi_2          0.222    0.009   24.490    0.000
   .f1_cdi_3          0.256    0.011   23.860    0.000
   .f1_cdi_4          0.212    0.008   25.543    0.000
   .f1_cdi_5          0.394    0.015   25.491    0.000
   .f1_cdi_6          0.264    0.011   23.011    0.000
   .f1_cdi_7          0.288    0.011   25.995    0.000
   .f1_cdi_8          0.297    0.011   25.832    0.000
   .f1_cdi_9          0.399    0.015   26.396    0.000
   .f1_cdi_10         0.350    0.014   25.529    0.000
   .f1_cdi_11         0.472    0.018   26.083    0.000
   .f1_cdi_12         0.285    0.012   24.491    0.000
    CDI_fu            0.220    0.015   14.479    0.000
```

In-Class Assignment

Try writing out an interpretation for …

- one factor loadings
- one variances of a deviance
- the variance of the latent variable

Scaling Constraints

By default lavaan fixes the first factor loading to 1 and estimates the variance for the latent factor.

We can change this by adding `std.lv = TRUE` to our cfa function.

Give that a try! Then try writing interpretations for the same parameter estimates that you looked at before. How do they differ?

Evaluating Models

Tools for Evaluating Models

We already know a variety of tools for evaluating models, and all of these apply for latent variable models…

- Residual Matrix (standardized) `lavResiduals()`
- Chi-square test of model fit (automatically output in summary)
- Fit indices (add `fit.measures = TRUE` to summary function)
  - RMSEA
  - SRMR
  - CFI
  - TLI

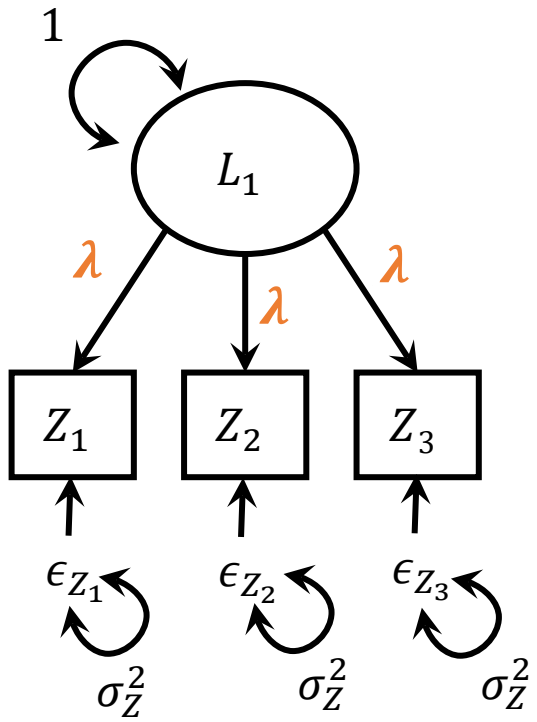
Take a look at all these for our current model, how do they look generally? Try to write a few sentences summarizing whether you feel the model has adequate fit.

Parallel Model

We can fit a parallel model to our data.

Remember this model forces the factor loadings and variances of the deviances to be equal across all items.

```
parallelsyntax <- '
  CDI_fu =~
lambda*f1_cdi_1+lambda*f1_cdi_2+lambda*f1_cdi_3+lambda*f1
_cdi_4+lambda*f1_cdi_5+lambda*f1_cdi_6+lambda*f1_cdi_7+la
mbda*f1_cdi_8+lambda*f1_cdi_9+lambda*f1_cdi_10+lambda*f1_
cdi_11+lambda*f1_cdi_12
  f1_cdi_1~~sigmaz*f1_cdi_1
  f1_cdi_2~~sigmaz*f1_cdi_2
  f1_cdi_3~~sigmaz*f1_cdi_3
  f1_cdi_4~~sigmaz*f1_cdi_4
  f1_cdi_5~~sigmaz*f1_cdi_5
  f1_cdi_6~~sigmaz*f1_cdi_6
  f1_cdi_7~~sigmaz*f1_cdi_7
  f1_cdi_8~~sigmaz*f1_cdi_8
  f1_cdi_9~~sigmaz*f1_cdi_9
  f1_cdi_10~~sigmaz*f1_cdi_10
  f1_cdi_11~~sigmaz*f1_cdi_11
  f1_cdi_12~~sigmaz*f1_cdi_12
'
parallelFitcfa <- cfa(parallelsyntax, data = cdidat,
std.lv = TRUE)
```



Evaluating Parallel Model

Take a look at our typical evaluation criteria for our current model, how do they look generally? Try to write a few sentences summarizing whether you feel the model has adequate fit.

$\chi^2(76) = 925.41, p < .001$ suggesting that the parallel model fits significantly worse than the saturated model. We would thus conclude that this model does not have perfect fit.

$CFI = 0.82, TLI = 0.84$ these fit indices suggest potential room for improvement, though the fit is not abysmal

$RMSEA = 0.09\ [90\%\ CI = 0.08{,}0.09, p < .001]$ This suggests that the RMSEA is significantly higher than the proposed cut-off of 0.05. The estimate is relatively precise (.01 CI width) and not too far from .05

$SRMR = .115$ similar to the other fit indices this suggests room for improvement, though it does not suggest very poor fit

A little aside on Model Syntax

Remember that you can specify model syntax in chunks. This can be helpful when fitting many similar models that trade out different chunks.

Parallel

```
factorloadingsfixed <- "CDI_fu =~
lambda*f1_cdi_1+lambda*f1_cdi_2+lambda*f1_cdi_3+lambda*f1_cdi_4+la
mbda*f1_cdi_5+lambda*f1_cdi_6+lambda*f1_cdi_7+lambda*f1_cdi_8+lamb
da*f1_cdi_9+lambda*f1_cdi_10+lambda*f1_cdi_11+lambda*f1_cdi_12"
variancesfixed <- "
 f1_cdi_1~~sigmaz*f1_cdi_1
  f1_cdi_2~~sigmaz*f1_cdi_2
  f1_cdi_3~~sigmaz*f1_cdi_3
  f1_cdi_4~~sigmaz*f1_cdi_4
  f1_cdi_5~~sigmaz*f1_cdi_5
  f1_cdi_6~~sigmaz*f1_cdi_6
  f1_cdi_7~~sigmaz*f1_cdi_7
  f1_cdi_8~~sigmaz*f1_cdi_8
  f1_cdi_9~~sigmaz*f1_cdi_9
  f1_cdi_10~~sigmaz*f1_cdi_10
  f1_cdi_11~~sigmaz*f1_cdi_11
  f1_cdi_12~~sigmaz*f1_cdi_12
"
```

Tau-Equivalent

Congeneric

```
factorloadingsfree <- "CDI_fu =~
f1_cdi_1+f1_cdi_2+f1_cdi_3+f1_cdi_4+f1_cdi_5+f1_cdi_6+f1_cdi_7+f1_
cdi_8+f1_cdi_9+f1_cdi_10+f1_cdi_11+f1_cdi_12"
```

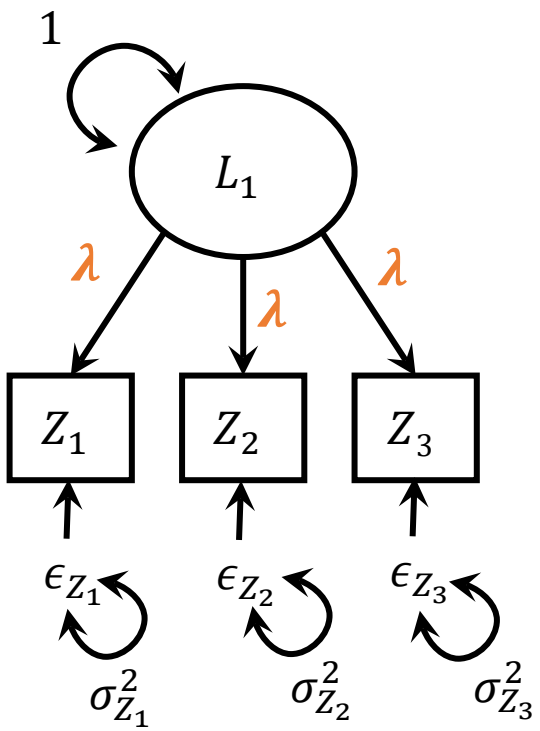
Tau-Equivalent Model

We can fit a tau-equivalent model to our data.

Remember this model forces the factor loadings to be equal across all items but the variances are allowed to differ.

```
taueqFitcfa <- cfa(factorloadingsfixed, data
= cdidat, std.lv = TRUE)
```

Take a look at our typical evaluation criteria for our current model, how do they look generally? Try to write a few sentences summarizing whether you feel the model has adequate fit.



Tau-Equivalent Summary

$\chi^2(65) = 515.91, p < .001$ suggesting that the tau-equivalent model fits significantly worse than the saturated model. We would thus conclude that this model does not have perfect fit.

$CFI = 0.90, TLI = 0.90$ these fit indices suggest fairly good fit, not perfect but not too shabby

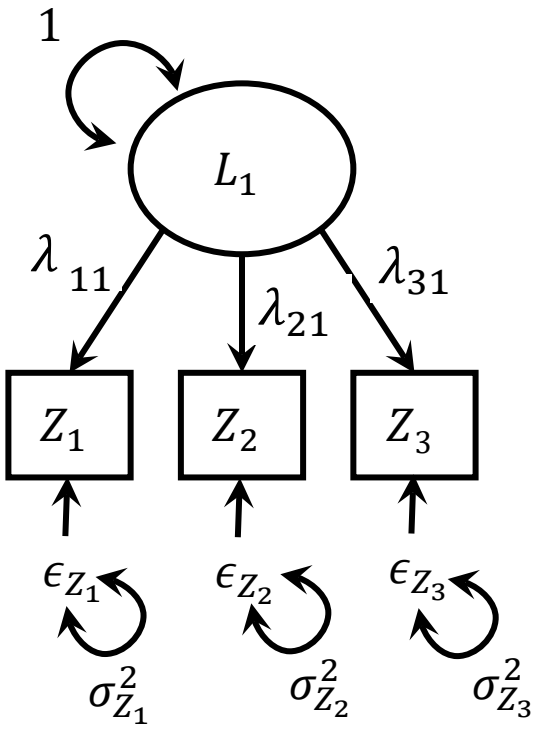
$RMSEA = 0.07\ [90\%\ CI = 0.06, 0.07, p < .001]$ This suggests that the RMSEA is significantly higher than the proposed cut-off of 0.05. The estimate is relatively precise (.01 CI width) but not too far from .05.

$SRMR = .09$ similar to the other fit indices this suggests room for improvement, though it does not suggest particularly bad fit

Congeneric Model

We've already fit the congeneric model, and evaluated it's fit.

Generally it looked pretty good. Not perfect, but not bad



Comparing Models

So far…

In looking at “comparing” the congeneric, parallel, and tau-equivalent models, we’ve bee able to evaluate each model individually, but haven’t compared them directly.

There are multiple methods for comparing models formally, and these should be used to compare candidate models.

Importantly, model comparison should really on occur among sets of reasonably well-performing models.

> Knowing that one bad model is better than another bad model doesn’t really help us.

Based on this, let’s compare the parallel, tau-equivalent, and congeneric models

Nested Models

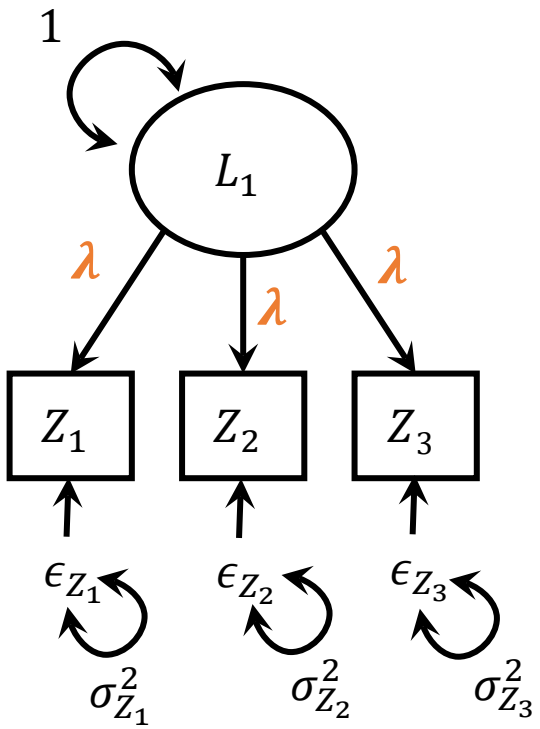
Consider two models: Model A and Model B

Model B is nested within Model A if…

Certain restrictions on Model A can lead us to Model B

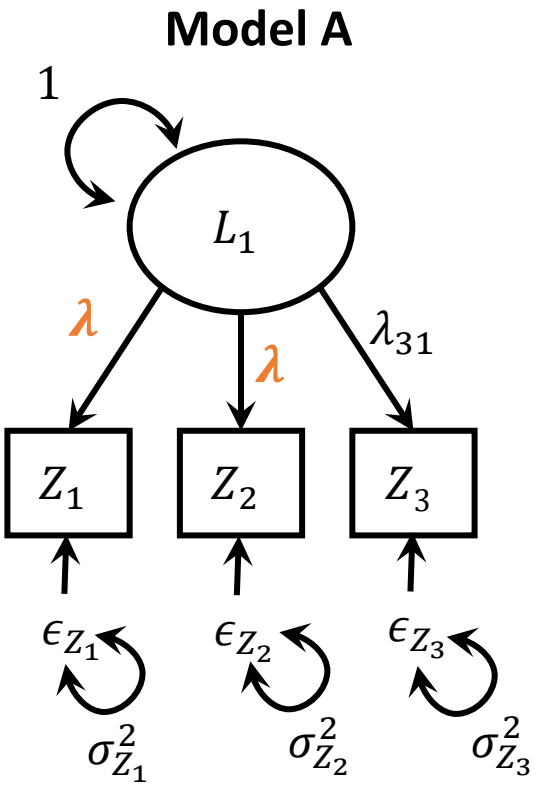
For example, the tau-equivalent model is nested within the congeneric model

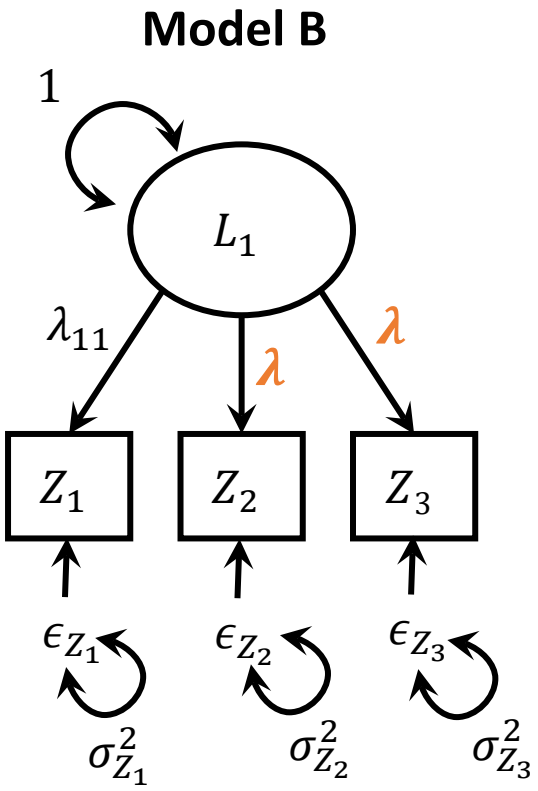
Similarly, the parallel model is nested within both the tau-equivalent model and the congeneric model



Non-nested Models

An example of a non-nested model





Likelihood Ratio Difference Test

The model χ^2 is a test of the fit difference between two models:

a) Saturated model with perfect fit and zero degrees of freedom

b) The fitted model with imperfect fit and degrees of freedom matching the number of constraints

These models are nested

We can test which models fits better by calculating

$$LR = -2[LL(\boldsymbol{\theta}_B) - 2LL(\boldsymbol{\theta}_A))$$

This statistic has a chi-square distribution with degrees of free equal to the difference in the degrees of freedom of the two models

$$\chi^2(df_B - df_A)$$

LRT in Lavaan

You can implement this LRT in lavaan using `lavtestLRT`

```
> lavTestLRT(measureFitcfapsivar, taueqFitcfa)
Chi-Squared Difference Test

                    Df   AIC   BIC  Chisq Chisq diff Df diff Pr(>Chisq)
measureFitcfapsivar 54 32102 32229 254.03
taueqFitcfa         65 32342 32411 515.91     261.88      11  < 2.2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

We take the difference between the two individual deviances for the models: $-2LL(\theta_A)$ and $-2LL(\theta_B)$ to get the difference

We take the difference between the two individual degrees of freedom for the models: df_A and df_B to get the difference

The p-value is calculated based on the chi-square and df

```
1-pchisq(261.88, 11)
```

Based on this test we would conclude that the congeneric model fits significantly better than the tau-equivalent model

Tau-Equivalent vs. Parallel

Try comparing the tau-equivalent model to the parallel model.

What do you find?

```
> lavTestLRT(taueqFitcfa, parallelFitcfa)
Chi-Squared Difference Test

               Df   AIC   BIC  Chisq Chisq diff Df diff Pr(>Chisq)
taueqFitcfa    65 32342 32411 515.91
parallelFitcfa 76 32729 32740 925.41      409.5      11  < 2.2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

The tau-equivalent model fits significantly better than the parallel model.

Where to start?

Given this process, you might wonder whether to start testing with most restrictive model or least restrictive

Parallel vs. Tau-Equivalent → Tau-Equivalent vs. Congeneric

Tau-Equivalent vs. Congeneric → Tau-Equivalent vs. Parallel

It is more typical to start with the least restrictive and then test whether restrictions **significantly reduce the fit of the model to the data**

Only consider models with reasonable fit on their own.

AIC and BIC

```
> lavTestLRT(measureFitcfapsivar, taueqFitcfa)
Chi-Squared Difference Test

                    Df   AIC   BIC  Chisq Chisq diff Df diff Pr(>Chisq)
measureFitcfapsivar 54 32102 32229 254.03
taueqFitcfa         65 32342 32411 515.91     261.88      11  < 2.2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

AIC: Akaike Information Criteria

$$-2LL(\theta) + 2t$$

BIC: Bayesian Information Criteria

$$-2LL(\theta) + t\log(N)$$

Used for model comparison in many contexts (not just SEM)

Each penalizes for complexity of the model slightly differently

BIC also accounts for sample size

Can be used for comparing nonnested models (yay!)

No formal hypothesis test, just “lower numbers are better”

Lagrangian Multiplier Tests (Modification Indices)

In some situations, you may find that your desired model does not fit well, and you want to explore alternative models that are very similar to your current model.

The basic idea: Compare to a model with a restricted parameter freed

See book for equations

```
modindices(taueqFitcfa)
```

By default, modification indices are printed out for each nonfree (or fixed-to-zero) parameter. The modification indices are supplemented by the expected parameter change (EPC) values (column `epc`). The last three columns contain the standardized EPC values (`sepc.lv`: only standardizing the latent variables; `sepc.all`: standardizing all variables; `sepc.nox`: standardizing all but exogenous observed variables).

Example with Tau-Equivalent Model

	lhs	op	rhs	mi	epc	sepc.lv	sepc.all	sepc.nox	
37	f1_cdi_1	~~	f1_cdi_2	12.730	0.025	0.025	0.103	0.103
38	f1_cdi_1	~~	f1_cdi_3	7.406	0.020	0.020	0.078	0.078
39	f1_cdi_1	~~	f1_cdi_4	0.295	0.004	0.004	0.016	0.016
40	f1_cdi_1	~~	f1_cdi_5	0.522	-0.006	-0.006	-0.020	-0.020
41	f1_cdi_1	~~	f1_cdi_6	10.760	0.026	0.026	0.093	0.093
42	f1_cdi_1	~~	f1_cdi_7	1.034	-0.008	-0.008	-0.029	-0.029
43	f1_cdi_1	~~	f1_cdi_8	19.982	-0.034	-0.034	-0.127	-0.127
44	f1_cdi_1	~~	f1_cdi_9	12.461	-0.031	-0.031	-0.099	-0.099
45	f1_cdi_1	~~	f1_cdi_10	1.257	0.009	0.009	0.032	0.032
46	f1_cdi_1	~~	f1_cdi_11	0.283	0.005	0.005	0.015	0.015
47	f1_cdi_1	~~	f1_cdi_12	24.201	0.038	0.038	0.140	0.140
48	f1_cdi_2	~~	f1_cdi_3	19.685	0.032	0.032	0.127	0.127
49	f1_cdi_2	~~	f1_cdi_4	9.503	-0.020	-0.020	-0.090	-0.090
50	f1_cdi_2	~~	f1_cdi_5	2.078	0.012	0.012	0.041	0.041
51	f1_cdi_2	~~	f1_cdi_6	15.087	0.029	0.029	0.111	0.111
52	f1_cdi_2	~~	f1_cdi_7	4.588	-0.015	-0.015	-0.061	-0.061
53	f1_cdi_2	~~	f1_cdi_8	0.145	-0.003	-0.003	-0.011	-0.011
54	f1_cdi_2	~~	f1_cdi_9	3.368	-0.015	-0.015	-0.052	-0.052
55	f1_cdi_2	~~	f1_cdi_10	17.531	-0.033	-0.033	-0.119	-0.119
56	f1_cdi_2	~~	f1_cdi_11	0.761	-0.008	-0.008	-0.024	-0.024
57	f1_cdi_2	~~	f1_cdi_12	3.158	-0.013	-0.013	-0.051	-0.051
58	f1_cdi_3	~~	f1_cdi_4	20.485	-0.031	-0.031	-0.131	-0.131
59	f1_cdi_3	~~	f1_cdi_5	7.578	0.025	0.025	0.077	0.077
60	f1_cdi_3	~~	f1_cdi_6	42.658	0.053	0.053	0.185	0.185
61	f1_cdi_3	~~	f1_cdi_7	3.029	-0.014	-0.014	-0.049	-0.049
62	f1_cdi_3	~~	f1_cdi_8	0.363	-0.005	-0.005	-0.017	-0.017
63	f1_cdi_3	~~	f1_cdi_9	0.804	-0.008	-0.008	-0.025	-0.025
64	f1_cdi_3	~~	f1_cdi_10	4.697	-0.019	-0.019	-0.061	-0.061
65	f1_cdi_3	~~	f1_cdi_11	0.147	-0.004	-0.004	-0.011	-0.011
66	f1_cdi_3	~~	f1_cdi_12	0.043	0.002	0.002	0.006	0.006
67	f1_cdi_4	~~	f1_cdi_5	0.032	-0.001	-0.001	-0.005	-0.005
68	f1_cdi_4	~~	f1_cdi_6	19.535	-0.032	-0.032	-0.127	-0.127

Some Useful Options

- Sorting the modification indices

  ```
  sort = TRUE
  ```

  - Will sort the modification indices from largest to smallest
- Print first N results

  ```
  maximum.number = N
  ```

  - Will print the first N indices
  - Good when used with sort
  - But also look at the whole list
- Looking at modifications of a certain type

  ```
  mi <- modindices(fit)
  mi[mi$op == "=~",]
  ```

  - Would show all modifications for loadings

Example with Tau-Equivalent Model

```
modindices(taueqFitcfa, sort = TRUE, maximum.number = 10)
```

	lhs	op	rhs	mi	epc	sepc.lv	sepc.all	sepc.nox
60	f1_cdi_3	~~	f1_cdi_6	42.658	0.053	0.053	0.185	0.185
47	f1_cdi_1	~~	f1_cdi_12	24.201	0.038	0.038	0.140	0.140
97	f1_cdi_9	~~	f1_cdi_10	21.513	0.047	0.047	0.128	0.128
93	f1_cdi_8	~~	f1_cdi_9	20.918	0.043	0.043	0.127	0.127
58	f1_cdi_3	~~	f1_cdi_4	20.485	-0.031	-0.031	-0.131	-0.131
43	f1_cdi_1	~~	f1_cdi_8	19.982	-0.034	-0.034	-0.127	-0.127
48	f1_cdi_2	~~	f1_cdi_3	19.685	0.032	0.032	0.127	0.127
68	f1_cdi_4	~~	f1_cdi_6	19.535	-0.032	-0.032	-0.127	-0.127
74	f1_cdi_4	~~	f1_cdi_12	19.152	0.031	0.031	0.126	0.126
55	f1_cdi_2	~~	f1_cdi_10	17.531	-0.033	-0.033	-0.119	-0.119

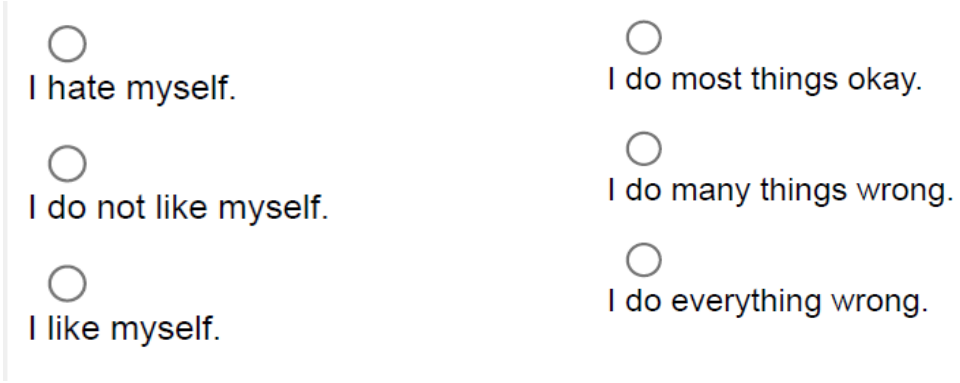
Can interpret these like a $\chi^2(1)$, remember under the null expected value of $\chi^2(df) = df$. So here 42 seems pretty large! (Twice as much as the next)

EPC is expected parameter change. Covariance would change from 0 to 0.053. Is that big? Hard to tell in covariance metric.

If all variables were standardized (i.e., var = 1), then the covariance would be a correlation. Correlation of 0.185 is non-trivial but also not huge.

Do you change the model?

- Go back to the items and evaluate whether a correlated residual makes sense



- Do you have other data you could test this relationship on, to evaluate whether it's generalizable past the sample?
- Ultimately the decision is up to you, but **transparency is key**. This is not something you hypothesized *a priori* and in your write up you might say something like…

> "Fit of our model was borderline, so we looked at the modification indices to evaluate whether any correlated residuals would help with model fit. The largest modification index was for Item 3 and 6 ($\chi^2(1) = 42.66$) so we added that correlated residual. The next largest modification index was about 24 with a steady decreasing distribution, so we did not consider additional modifications."

What happens?

```
mod1 <- "f1_cdi_3~~f1_cdi_6"
taueqFitcfa2 <- cfa(c(factorloadingsfixed, mod1), data = cdidat,
std.lv = TRUE)
summary(taueqFitcfa2, fit.measures=TRUE)
lavTestLRT(measureFitcfapsivar, taueqFitcfa2)
```

	Tau-Equivalent	Tau-Equivalent+Mod
Chi-square	515.909	473.485
df	65	64
CFI	.904	0.913
TLI	.903	0.911
RMSEA	.068	0.066
SRMR	.090	0.086
P-value comparison to congeneric	<.001	<.001

Whack-a-Mole and Rabbit Hole

Modification indices are a strong and powerful tool, but they have limitations:

- If your misspecification is “broader” than a single parameter you can end up in a WHACK-A-MOLE situation.
  - Adjust a parameter over here, now the problem pops up somewhere else
- It can be tempting to wander down the rabbit hole
  - Continuously editing a model then looking at the modification indices, rinse and repeat until you have a good fitting model.
  - You may find yourself in a very different place than you started.